EuroMLSys' 26

Why Smaller is Slower?

Dimensional Misalignment in Compressed LLMs

Jihao Xin, Tian Lyu, Qilong Pan, Kesen Wang, Marco Canini

KAUST

HUMAIN

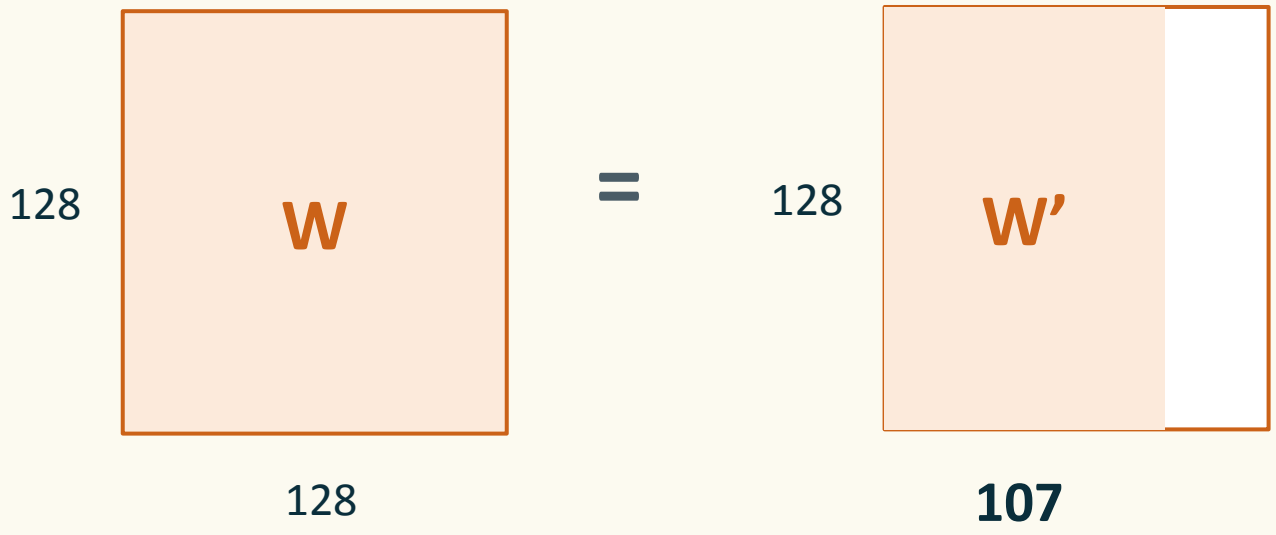
Background: Post-training compression in LLMs

- Large pre-trained models such as GPTs incur substantial memory usage.
- Post-training compression is proposed to reduce the model size.
- Popular approaches include:

1. **Low-rank Decomposition**
2. **Structured Pruning**
3. **Token Eviction**

Problem: Post-training Compression produces irregular tensor dimensions

- LLM dimensions are carefully designed to fit optimized GPU kernels



Smaller models might be slower!!!

Parameters

−15%

Prefill latency

+1%

ρ = 15%, A = 100, S = 1024
ASVD [1] on Llama-3-8B

Dimensional Misalignment:

Irregular dimension degrades inference performance on GPUs

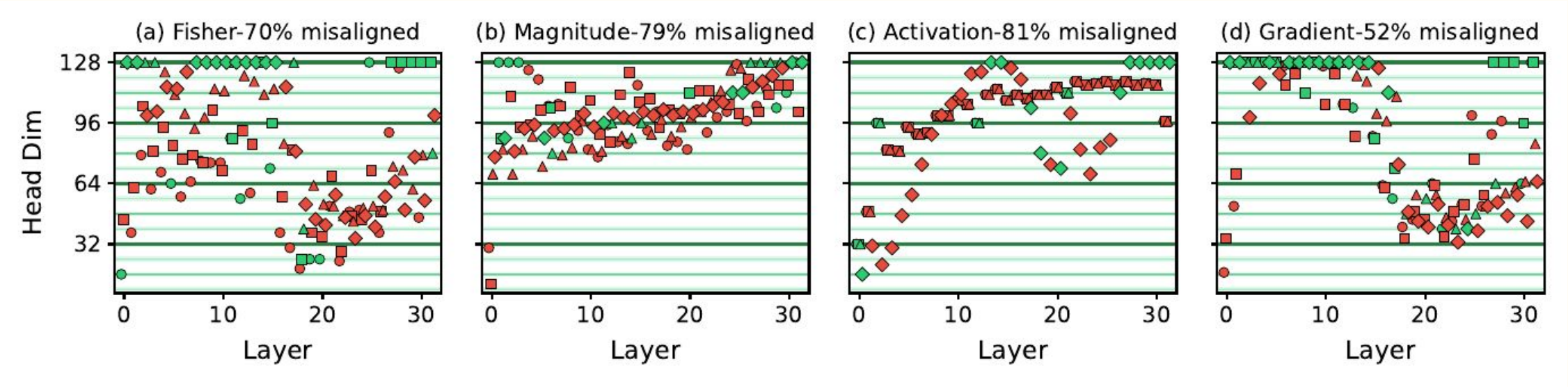
[1] Yuan, Zhihang, et al. "Asvd: Activation-aware singular value decomposition for compressing large language models." *arXiv preprint arXiv:2312.05821* (2023).

Investigation: Dimensional misalignment is a common problem in compressed LLMs

Consistently observed across different *compressors*, *compression ratios*, *budget allocation strategies*

Example: Llama -3-8B at 20% compression.

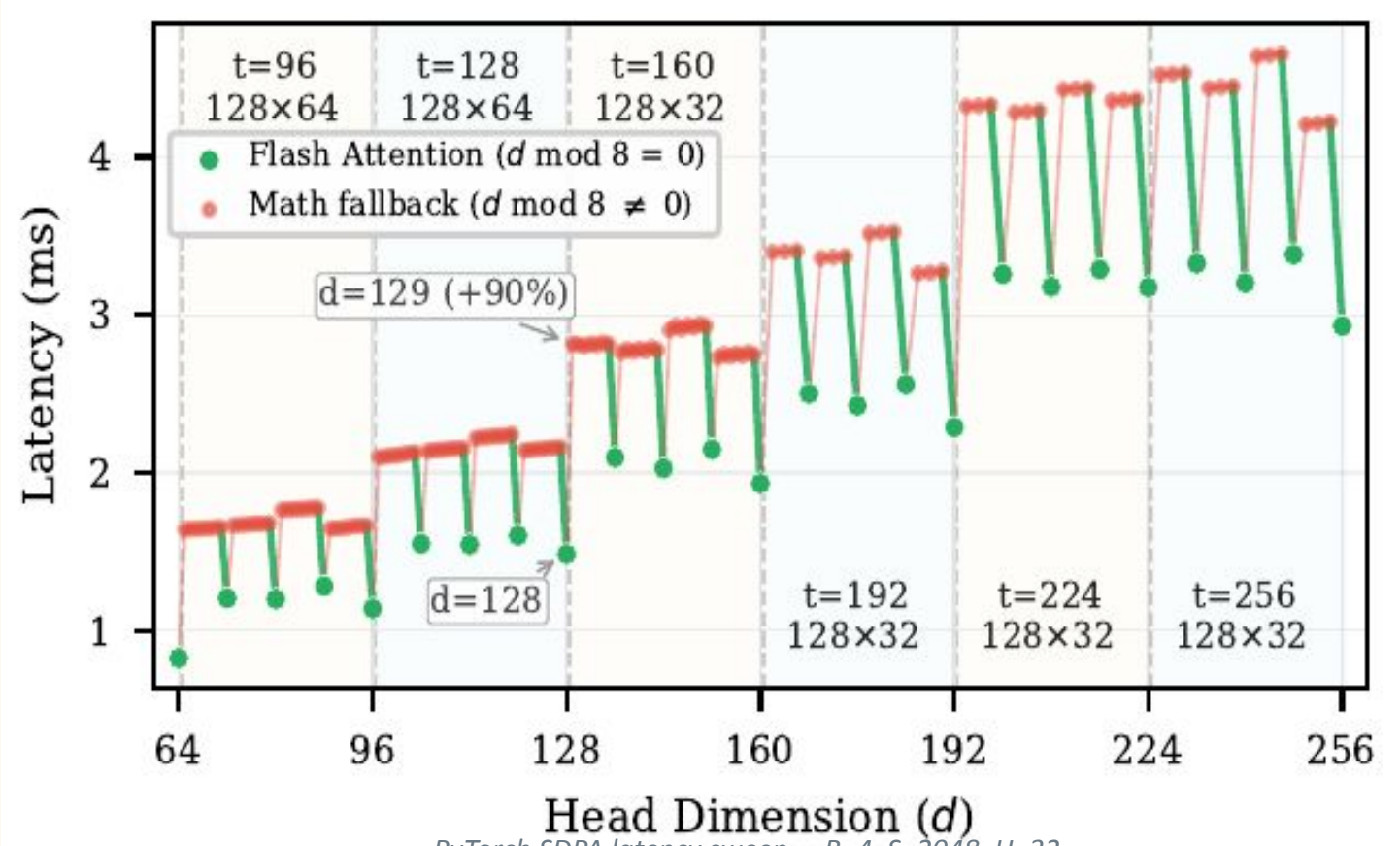
Aligned (d%8 ==0) Misaligned (d%8 != 0)



● Q ▪ K ▲ V

Why is dimensional misalignment a problem?

- Exemplified by SDPA(Scaled Dot Product Attention), a key kernel in LLMs
- PyTorch uses **Flash Attention Backend** only if dims are 8-aligned
- When not 8-aligned, PyTorch falls back to naive **Math Backend**



PyTorch SDPA latency sweep · B=4, S=2048, H=32

Backend fallback

+90% latency

Full Stack Root Cause Analysis

Framework
FlashAttn / Math

worst case

+90%

→

Library
cuBLAS / CUTLASS

worst case

+30%

→

Hardware
Tensor Core / L2 Cache

worst case

+70%

Alignment Constraints Table

●	**Framework**	SDPA backend	`d % 8 = 0`	**~90%**
●	**Framework**	FA2 template	`d % 32 = 0`	**~30%**
●	**Library**	cuBLAS GEMM	`K, N % 8 = 0`	**~30%**
●	**Hardware**	Tensor Core + L2	`K % 16 = 0`	**~70%**

Our Proposal: GAC:GPU-Aligned Compression

Not a compressor, but a new compression paradigm.

I

Input

Model

Compressor

Budget

→

1

Unconstrained Compression

Run existing compressors as-is

→

2

Dimensions Sweep

Profile kernels

Generate dim candidates

→

3

Dimension Adjustment

Constrained Optimization

DP Solver

✓ **Output: Compressed model with 100%-aligned dims, guaranteed speedup.**

Step 3: Dimension Adjustment

Constrained Optimization

Input:

- LLM weights {w}
- Each weight has an importance score
- Each weight has a candidate dim set {c}
- Compression Budget

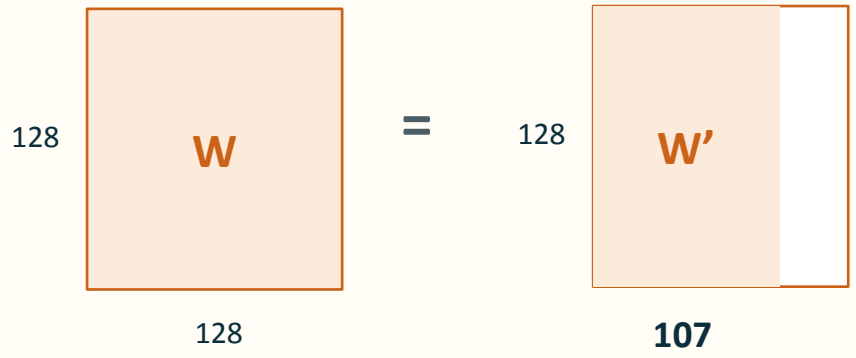
Target: Maximize total importance score

Constraints:

- total params ≤ compression budget
- only choose from candidate set

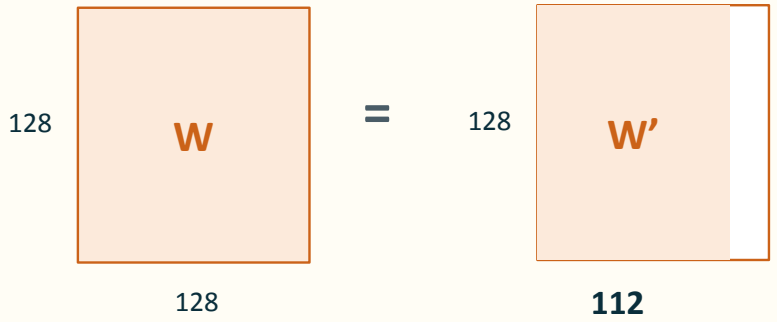
💡 *Multi-Choice Knapsack Problem – DP Solver*

< 1 s **No runtime overhead**



❌ **107 is not an 8-aligned dim**

✅ **C_i = {96, 104, 112✅, 128}**



Preliminary Evaluation

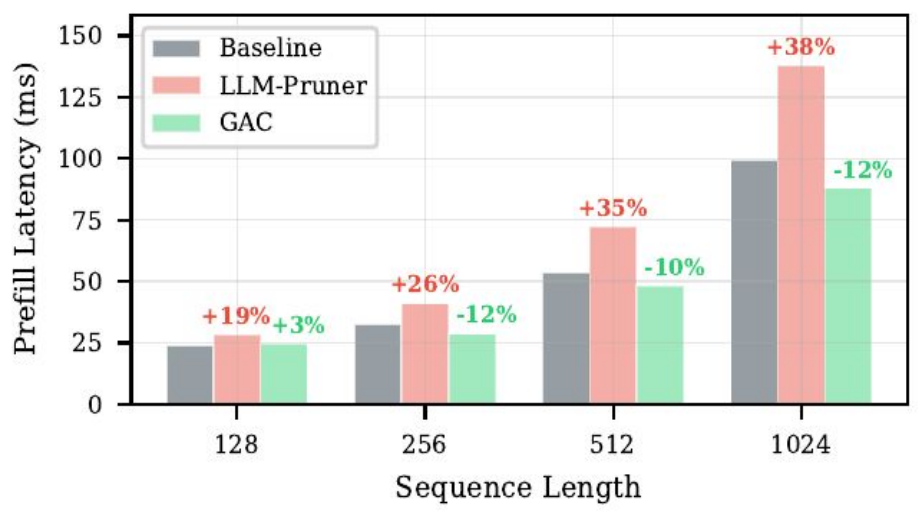
100%
Alignment

Up to **1.5×**
Prefill speedup

Preserve **Accuracy**
PPL & Zero-shot metric

Llama-3-8B, ρ = 15%, A=100, B=1, S=1024, prefill

Method	Align	PPL	Lat (ms)	Speedup
Baseline	100%	6.14	99.6	—
ASVD	5%	34.7	100.5	-1%
ASVD (GAC)	100%	31.3	67.1	+49%
LLM-Pruner	83%	9.88	137.7	-38%
LLM-Pruner (GAC)	100%	9.87	88.0	+12%



Why Smaller Is Slower?

Dimensional Misalignment in Compressed LLMs

Summary

Problem

Dimensional Misalignment
in compressed models

Analysis

Root Cause
across 3 layers.

Solution

GAC
GPU-Aligned Compression

Thank you · Questions?
github.com/sands-lab/GAC

KAUST HUMAIN